



جامعة دمشق
كلية الهندسة المعلوماتية

تقرير وظيفة البرمجة التفرعية
محرك المعالجة عالي الأداء لنظام التجارة الالكترونية

High-Performance E-Commerce Backend Engine

إعداد الطلاب

أمل محمد شعبان

عائشة محمد حسام الدالاتي

هايدي محمد خير مرعك

نور محمد الإمام

المقدمة

يهدف هذا المشروع إلى تطبيق مجموعة من المتطلبات غير الوظيفية على نظام تجارة إلكترونية مبسط مبني باستخدام Django. لا يركز المشروع على بناء متجر إلكتروني كامل بواجهة مستخدم نهائية، وإنما يركز على دراسة سلوك النظام من ناحية الأداء، التزامن، إدارة الموارد، المعالجة غير المتزامنة، المعالجة الدفعية، وتوزيع الحمل.

تم بناء النظام حول فكرة E-Commerce بسيطة تحتوي على منتجات، مخزون، طلبات شراء، وعناصر طلب. هذه الوظائف الأساسية استخدمت كأساس لتجربة المتطلبات غير الوظيفية المطلوبة. على سبيل المثال، تم استخدام عملية شراء منتج من المخزون لاختبار مشكلة Race Condition، وتم استخدام عملية دفع محاكية لاختبار إدارة الموارد، كما تم استخدام إرسال إيميل وتوليد فاتورة بشكل محاكي لاختبار المعالجة غير المتزامنة.

بعض العمليات في المشروع لم تُنفذ بخدمات حقيقية، بل تمت محاكاتها بشكل مقصود. فمثلاً، عملية الدفع لا ترتبط ببوابة دفع حقيقية مثل Stripe أو PayPal، وإرسال الإيميل وتوليد الفاتورة لا يستخدمان خدمات خارجية حقيقية. تم اعتماد هذه المحاكاة لأن الهدف الأساسي من المشروع هو اختبار المتطلبات غير الوظيفية وليس بناء نظام إنتاجي كامل أو ربطه بخدمات خارجية.

يتضمن المشروع خمسة متطلبات غير وظيفية رئيسية. المتطلب الأول يعالج مشكلة Race Condition عند وصول عدة طلبات شراء إلى نفس المنتج في نفس الوقت. المتطلب الثاني يركز على إدارة الموارد باستخدام Thread Pool للحد من عدد عمليات الدفع الثقيلة التي تعمل بالتوازي. المتطلب الثالث يطبق فكرة Asynchronous Queue لنقل المهام الثانوية خارج مسار الاستجابة الرئيسي. المتطلب الرابع يطبق Batch Processing لمعالجة المبيعات اليومية على دفعات. أما المتطلب الخامس فيطبق Load Distribution لتوزيع الطلبات على أكثر من نسخة من التطبيق.

تم اختبار كل متطلب عملياً باستخدام سكريبتات Python أو JMeter أو أوامر Django management commands، وتم توثيق النتائج باستخدام لقطات شاشة توضح الفرق بين الحالة قبل تطبيق الحل وبعده. بناءً على ذلك، يوضح التقرير ليس فقط الكود الذي تم تنفيذه، بل أيضاً سبب اختيار كل حل، وطريقة اختبارها، والنتائج التي تثبت فعاليتها.

نظرة عامة على المشروع

يعتمد المشروع على نظام تجارة إلكترونية مبسط تم تطويره باستخدام Django. تم اختيار هذا النوع من الأنظمة لأنه يحتوي على عمليات مناسبة لاختبار المتطلبات غير الوظيفية، مثل شراء المنتجات، خصم المخزون، إنشاء الطلبات، تنفيذ الدفع، إرسال الإشعارات، توليد الفواتير، ومعالجة تقارير المبيعات.

يتكون المشروع بشكل رئيسي من تطبيق **products** وتطبيق **orders**. يحتوي تطبيق **products** على المنتجات والمخزون، بينما يحتوي تطبيق **orders** على الطلبات، عناصر الطلب، السلة، إضافة إلى أجزاء خاصة ببعض المتطلبات مثل سجلات المعالجة الدفعية.

تم ربط كل متطلب غير وظيفي بسيناريو مناسب داخل نظام التجارة الإلكترونية. فمثلاً، استخدمنا شراء المنتج لاختبار Race Condition، واستخدمنا عملية دفع محاكية لاختبار إدارة الموارد، واستخدمنا إرسال إيميل وتوليد فاتورة لاختبار المعالجة غير المتزامنة، واستخدمنا جرد المبيعات اليومية لاختبار Batch Processing، ثم استخدمنا أكثر من نسخة من التطبيق لاختبار Load Distribution.

بعض العمليات تم تنفيذها فعلياً داخل قاعدة البيانات، مثل إنشاء الطلبات وخصم المخزون. أما عمليات أخرى مثل الدفع، إرسال الإيميل، وتوليد الفاتورة فقد تمت محاكاتها، لأن الهدف الأساسي من المشروع هو اختبار السلوك غير الوظيفي وليس ربط النظام بخدمات خارجية حقيقية.

المتطلبات الوظيفية Functional Requirements

رغم أن تركيز المشروع الأساسي هو المتطلبات غير الوظيفية، كان لا بد من وجود وظائف أساسية تمثل نظام تجارة إلكترونية مبسط. هذه الوظائف لم تبني كنظام متجر كامل، بل كقاعدة عملية لاختبار المتطلبات المطلوبة.

1. المنتجات

يحتوي المشروع على نموذج **Product** الذي يمثل المنتج داخل النظام. يتضمن المنتج اسماً وسعراً، ويتم استخدامه في عمليات الشراء وإنشاء الطلبات التجريبية.

2. المخزون

يمثل نموذج **Stock** كمية المنتج المتاحة في المخزون. استخدمنا هذا الجزء بشكل أساسي في اختبار Race Condition، حيث يتم إرسال عدة طلبات شراء متزامنة على نفس المنتج للتحقق من سلامة تحديث المخزون.

3. الطلبات

يمثل نموذج **Order** طلب الشراء داخل النظام. يتم إنشاء الطلب عند نجاح عملية الشراء، كما استخدمت الطلبات لاحقاً في اختبار المعالجة الدفعية من خلال توليد عدد كبير من الطلبات المكتملة.

4. عناصر الطلب

يمثل نموذج **OrderItem** المنتجات الموجودة داخل كل طلب، مع الكمية والسعر. هذا النموذج يربط الطلب بالمنتجات التي تم شراؤها، ويجعل عملية الشراء أوضح داخل قاعدة البيانات.

5. السلة

يحتوي المشروع أيضاً على نماذج **Cart** و **CartItem** كسلة تسوق مبدئية. لم تكن السلة محور الاختبارات، لكنها جزء طبيعي من بنية نظام التجارة الإلكترونية.

6. العمليات المحاكاة

بعض الوظائف لم تُنفذ تخدمات حقيقية، بل تمت محاكاتها لخدمة الاختبارات. عملية الدفع تمت محاكاتها لاختبار إدارة الموارد، بينما تمت محاكاة إرسال الإيميل وتوليد الفاتورة لاختبار المعالجة غير المتزامنة. هذا الاختيار كان لأن الهدف من المشروع ليس بناء نظام دفع أو بريد إلكتروني حقيقي، بل اختبار تأثير هذه العمليات على الأداء وزمن الاستجابة واستهلاك الموارد.

7. جرد المبيعات اليومية

تم تنفيذ وظيفة جرد المبيعات اليومية في متطلب Batch Processing. تقوم هذه الوظيفة بمعالجة الطلبات المكتملة على دفعات، وحساب الإيرادات، وتسجيل نتائج كل تشغيل وكل دفعة.

8. توزيع الطلبات

في متطلب Load Distribution تم تنفيذ Load Balancer بسيط يوزع الطلبات على أكثر من نسخة من التطبيق. هذه الوظيفة تساعد على توضيح كيف يمكن تقليل الضغط على نسخة واحدة من التطبيق من خلال توزيع الحمل.

9. حدود المتطلبات الوظيفية

المشروع لا يحتوي على واجهة مستخدم كاملة أو نظام دفع حقيقي أو إرسال بريد إلكتروني فعلي. تم تنفيذ الوظائف الأساسية فقط بالقدر الذي يسمح باختبار المتطلبات غير الوظيفية المطلوبة بشكل واضح وعملي.

المتطلبات غير الوظيفية Non-Functional Requirements

يركز هذا القسم على المتطلبات غير الوظيفية التي تم تطبيقها في المشروع. تم اختيار كل متطلب بناءً على مشكلة عملية يمكن أن تظهر في أنظمة التجارة الإلكترونية، ثم تم تطبيق حل مناسب واختباره عملياً. الهدف من هذا القسم هو توضيح المشكلة، الحل المطبق، وطريقة التحقق من أن الحل أعطى النتيجة المطلوبة.

1. Race Condition Handling

تظهر مشكلة Race Condition عندما يحاول أكثر من طلب الوصول إلى نفس البيانات وتعديلها في نفس الوقت. في مشروعنا تظهر هذه المشكلة عند محاولة عدة مستخدمين شراء نفس المنتج بالتزامن. إذا قرأت عدة طلبات نفس قيمة المخزون قبل تحديثها، يمكن أن يقبل النظام عدد طلبات أكبر من الكمية المتوفرة فعلياً.

لحل هذه المشكلة كانت هناك عدة حلول ممكنة، مثل استخدام Optimistic Locking، أو Distributed Locks، أو Pessimistic Locking. في هذا المشروع اخترنا استخدام Pessimistic Locking على مستوى قاعدة البيانات، وذلك باستخدام:

```
transaction.atomic()
```

```
select_for_update()
```

تضمن `transaction.atomic()` أن عملية الشراء تتم كوحدة واحدة، بينما يقوم `select_for_update()` بقفل صف المخزون أثناء تنفيذ الطلب،

بحيث لا يستطيع طلب آخر تعديله في نفس اللحظة.

تم تنفيذ حالتين للاختبار.

الحالة الأولى هي النسخة غير الآمنة `/orders/place-order-unsafe/`،

والحالة الثانية هي النسخة الآمنة `/orders/place-order/`.

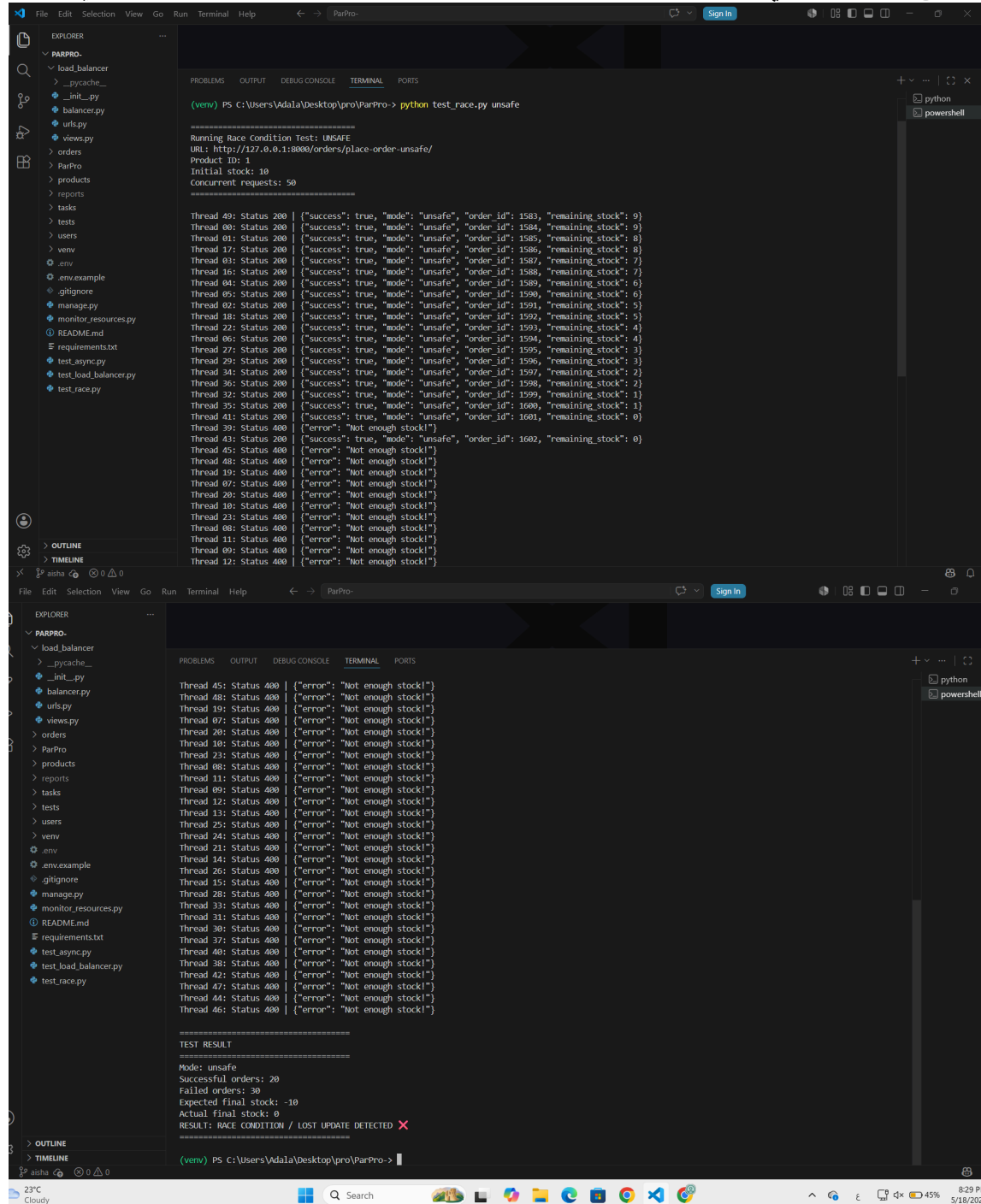
تم اختبار الحالتين باستخدام الملف:

```
test_race.py
```

حيث تم إرسال 50 طلباً متزامناً على منتج يحتوي على مخزون ابتدائي مقداره 10.

Unsafe Race Condition Result

أظهرت نتيجة النسخة غير الآمنة أن عدد الطلبات الناجحة كان أكبر من المخزون المتاح، كما ظهر اختلاف بين المخزون المتوقع والمخزون الفعلي. هذا يدل على حدوث Lost Update بسبب الوصول المتزامن غير المحمي.



```
File Edit Selection View Go Run Terminal Help
ParPro-
Sign In

EXPLORER
  PARPRO-
    load_balancer
      __pycache__
      __init__.py
      balancer.py
      urls.py
      views.py
    orders
    ParPro
    products
    reports
    tasks
    tests
    users
    venv
    .env
    .env.example
    .gitignore
    manage.py
    monitor_resources.py
    README.md
    requirements.txt
    test_async.py
    test_load_balancer.py
    test_race.py

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python test_race.py unsafe

Running Race Condition Test: UNSAFE
URL: http://127.0.0.1:8000/orders/place-order-unsafe/
Product ID: 1
Initial stock: 10
Concurrent requests: 50

Thread 40: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1583, "remaining_stock": 9}
Thread 00: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1584, "remaining_stock": 9}
Thread 01: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1585, "remaining_stock": 8}
Thread 17: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1586, "remaining_stock": 8}
Thread 03: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1587, "remaining_stock": 7}
Thread 16: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1588, "remaining_stock": 7}
Thread 04: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1589, "remaining_stock": 6}
Thread 05: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1590, "remaining_stock": 6}
Thread 02: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1591, "remaining_stock": 5}
Thread 18: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1592, "remaining_stock": 5}
Thread 22: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1593, "remaining_stock": 4}
Thread 06: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1594, "remaining_stock": 4}
Thread 27: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1595, "remaining_stock": 3}
Thread 29: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1596, "remaining_stock": 3}
Thread 34: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1597, "remaining_stock": 2}
Thread 36: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1598, "remaining_stock": 2}
Thread 32: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1599, "remaining_stock": 1}
Thread 35: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1600, "remaining_stock": 1}
Thread 41: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1601, "remaining_stock": 0}
Thread 39: Status 400 | {"error": "not enough stock!"}
Thread 43: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1602, "remaining_stock": 0}
Thread 45: Status 400 | {"error": "not enough stock!"}
Thread 42: Status 400 | {"error": "not enough stock!"}
Thread 19: Status 400 | {"error": "not enough stock!"}
Thread 07: Status 400 | {"error": "not enough stock!"}
Thread 20: Status 400 | {"error": "not enough stock!"}
Thread 10: Status 400 | {"error": "not enough stock!"}
Thread 23: Status 400 | {"error": "not enough stock!"}
Thread 08: Status 400 | {"error": "not enough stock!"}
Thread 13: Status 400 | {"error": "not enough stock!"}
Thread 09: Status 400 | {"error": "not enough stock!"}
Thread 12: Status 400 | {"error": "not enough stock!"}
Thread 13: Status 400 | {"error": "not enough stock!"}
Thread 25: Status 400 | {"error": "not enough stock!"}
Thread 24: Status 400 | {"error": "not enough stock!"}
Thread 21: Status 400 | {"error": "not enough stock!"}
Thread 14: Status 400 | {"error": "not enough stock!"}
Thread 26: Status 400 | {"error": "not enough stock!"}
Thread 15: Status 400 | {"error": "not enough stock!"}
Thread 28: Status 400 | {"error": "not enough stock!"}
Thread 33: Status 400 | {"error": "not enough stock!"}
Thread 31: Status 400 | {"error": "not enough stock!"}
Thread 30: Status 400 | {"error": "not enough stock!"}
Thread 27: Status 400 | {"error": "not enough stock!"}
Thread 40: Status 400 | {"error": "not enough stock!"}
Thread 38: Status 400 | {"error": "not enough stock!"}
Thread 42: Status 400 | {"error": "not enough stock!"}
Thread 47: Status 400 | {"error": "not enough stock!"}
Thread 44: Status 400 | {"error": "not enough stock!"}
Thread 46: Status 400 | {"error": "not enough stock!"}

TEST RESULT
Mode: unsafe
Successful orders: 20
Failed orders: 30
Expected final stock: -10
Actual final stock: 0
RESULT: RACE CONDITION / LOST UPDATE DETECTED ✖

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
```

Safe Transaction Result

بعد تطبيق القفل باستخدام `select_for_update()`، نجح فقط عدد الطلبات المطابق للمخزون المتاح، وفشلت باقي الطلبات برسالة عدم كفاية المخزون. كما أصبح المخزون الفعلي مطابقا للمخزون المتوقع، مما يثبت أن المشكلة تم حلها.

```
File Edit Selection View Go Run Terminal Help
ParPro

EXPLORER
  PARPRO-
    load_balancer
      __pycache__
        __init__.py
        balancer.py
        urls.py
        views.py
      orders
      ParPro
      products
      reports
      tasks
      tests
      users
      venv
      .env
      .env.example
      .gitignore
      manage.py
      monitor_resources.py
      README.md
      requirements.txt
      test_async.py
      test_load_balancer.py
      test_race.py

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python test_race.py safe

Running Race Condition Test: SAFE
URL: http://127.0.0.1:8000/orders/place-order/
Product ID: 1
Initial stock: 10
Concurrent requests: 50

Thread 49: Status 200 | {"success": true, "mode": "safe", "order_id": 1603}
Thread 01: Status 200 | {"success": true, "mode": "safe", "order_id": 1604}
Thread 46: Status 200 | {"success": true, "mode": "safe", "order_id": 1605}
Thread 48: Status 200 | {"success": true, "mode": "safe", "order_id": 1606}
Thread 44: Status 200 | {"success": true, "mode": "safe", "order_id": 1607}
Thread 40: Status 200 | {"success": true, "mode": "safe", "order_id": 1608}
Thread 00: Status 200 | {"success": true, "mode": "safe", "order_id": 1609}
Thread 43: Status 200 | {"success": true, "mode": "safe", "order_id": 1610}
Thread 12: Status 200 | {"success": true, "mode": "safe", "order_id": 1611}
Thread 06: Status 200 | {"success": true, "mode": "safe", "order_id": 1612}
Thread 18: Status 400 | {"error": "Not enough stock!"}
Thread 15: Status 400 | {"error": "Not enough stock!"}
Thread 16: Status 400 | {"error": "Not enough stock!"}
Thread 13: Status 400 | {"error": "Not enough stock!"}
Thread 17: Status 400 | {"error": "Not enough stock!"}
Thread 05: Status 400 | {"error": "Not enough stock!"}
Thread 22: Status 400 | {"error": "Not enough stock!"}
Thread 19: Status 400 | {"error": "Not enough stock!"}
Thread 23: Status 400 | {"error": "Not enough stock!"}
Thread 20: Status 400 | {"error": "Not enough stock!"}
Thread 11: Status 400 | {"error": "Not enough stock!"}
Thread 08: Status 400 | {"error": "Not enough stock!"}
Thread 03: Status 400 | {"error": "Not enough stock!"}
Thread 09: Status 400 | {"error": "Not enough stock!"}
Thread 34: Status 400 | {"error": "Not enough stock!"}
Thread 29: Status 400 | {"error": "Not enough stock!"}
Thread 28: Status 400 | {"error": "Not enough stock!"}
Thread 30: Status 400 | {"error": "Not enough stock!"}
Thread 31: Status 400 | {"error": "Not enough stock!"}
Thread 27: Status 400 | {"error": "Not enough stock!"}
Thread 32: Status 400 | {"error": "Not enough stock!"}
Thread 33: Status 400 | {"error": "Not enough stock!"}
Thread 35: Status 400 | {"error": "Not enough stock!"}
Thread 36: Status 400 | {"error": "Not enough stock!"}
Thread 37: Status 400 | {"error": "Not enough stock!"}
Thread 38: Status 400 | {"error": "Not enough stock!"}
Thread 39: Status 400 | {"error": "Not enough stock!"}
Thread 41: Status 400 | {"error": "Not enough stock!"}
Thread 42: Status 400 | {"error": "Not enough stock!"}
Thread 45: Status 400 | {"error": "Not enough stock!"}
Thread 47: Status 400 | {"error": "Not enough stock!"}

TEST RESULT
=====
Mode: safe
Successful orders: 10
Failed orders: 40
Expected final stock: 0
Actual final stock: 0
RESULT: CONSISTENT ✓

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
```

Resource Management and Capacity Control.2

في بعض أنظمة التجارة الإلكترونية توجد عمليات ثقيلة مثل معالجة الدفع. إذا تم تنفيذ كل عملية دفع مباشرة بدون أي تحكم بعدد العمليات المتزامنة، فقد يؤدي ذلك إلى استهلاك كبير للذاكرة والمعالج، خصوصاً عند وجود عدد كبير من الطلبات في نفس الوقت.

الحلول الممكنة لهذه المشكلة تشمل إنشاء Thread لكل طلب، أو استخدام Cached Thread Pool، أو استخدام Fixed Thread Pool. اخترنا في المشروع استخدام Fixed Thread Pool لأنه يسمح بتحديد عدد ثابت من العمليات التي تعمل بالتوازي، ويمنع تشغيل عدد غير محدود من المهام الثقيلة.

تم تنفيذ حالتين:

`/orders/process – payment – uncontrolled/`

`/orders/process – payment – controlled/`

في الحالة الأولى يتم تنفيذ عملية الدفع مباشرة مع كل طلب. أما في الحالة الثانية فيتم تمرير عمليات الدفع إلى `ThreadPoolExecutor` بعدد `workers` محدود، وهو 5 `workers` في مشروعنا.

تم اختبار المتطلب باستخدام JMeter لإرسال عدد كبير من الطلبات المتزامنة، بالإضافة إلى ملف `monitor_resources.py` لمراقبة استهلاك CPU و RAM. كما تم استخدام endpoint خاص لقراءة عدد عمليات الدفع المتزامنة:

`/orders/payment-metrics/`

في الحالة غير المضبوطة ظهر أن عدد عمليات الدفع المتزامنة وصل إلى عدد كبير من الطلبات، مثل 50 عملية متزامنة. هذا يوضح أن النظام يسمح بتشغيل جميع العمليات الثقيلة في نفس الوقت.

```
Label: uncontrolled_50
Output CSV: reports\resource_uncontrolled_50_20260518_210248.csv
CPU warning limit: 90.0%
RAM warning limit: 85.0%
Press CTRL + C to stop monitoring.

[2026-05-18 21:02:49] CPU: 1.9% | RAM: 73.1%
[2026-05-18 21:02:50] CPU: 4.8% | RAM: 73.2%
[2026-05-18 21:02:51] CPU: 6.5% | RAM: 73.3%
[2026-05-18 21:02:52] CPU: 17.0% | RAM: 80.4%
[2026-05-18 21:02:53] CPU: 3.3% | RAM: 81.1%
[2026-05-18 21:02:54] CPU: 1.0% | RAM: 81.1%
[2026-05-18 21:02:55] CPU: 4.3% | RAM: 81.1%
[2026-05-18 21:02:56] CPU: 4.1% | RAM: 81.1%
[2026-05-18 21:02:57] CPU: 5.1% | RAM: 80.8%
[2026-05-18 21:02:58] CPU: 5.8% | RAM: 78.6%
[2026-05-18 21:02:59] CPU: 8.0% | RAM: 74.2%
[2026-05-18 21:03:00] CPU: 5.7% | RAM: 73.4%
[2026-05-18 21:03:01] CPU: 2.6% | RAM: 73.3%
[2026-05-18 21:03:02] CPU: 5.2% | RAM: 73.3%
[2026-05-18 21:03:03] CPU: 4.4% | RAM: 73.7%
[2026-05-18 21:03:04] CPU: 3.9% | RAM: 73.9%

Monitoring stopped.
Results saved to: reports\resource_uncontrolled_50_20260518_210248.csv
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro> (Invoke-WebRequest -uri "http://127.0.0.1:8000/orders/payment-metrics/" -Method GET).metrics

uncontrolled_active      : 0
uncontrolled_max_active  : 40
uncontrolled_total_requests : 40
controlled_active        : 0
controlled_max_active    : 0
controlled_total_requests : 0

(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
```


resource_management_test.jmx (C:\Users\Adala\Desktop\pro\ParPro-tests\jmeter\resource_management_test.jmx) - Apache JMeter (5.6.3)

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse...

Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
POST uncon...	50	5833	1	8004	2935.28	20.00%	6.0/sec	6.22	1.10	1069.8
TOTAL	50	5833	1	8004	2935.28	20.00%	6.0/sec	6.22	1.10	1069.8

☐ Include group name in label? ☒ Save Table Header

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
 (venv) PS C:\Users\Adala\Desktop\pro\ParPro->
 (venv) PS C:\Users\Adala\Desktop\pro\ParPro->

resource_management_test.jmx (C:\Users\Adala\Desktop\pro\ParPro-tests\jmeter\resource_management_test.jmx) - Apache JMeter (5.6.3)

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename: Browse...

Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughp...	Received ...	Sent KB/s...
POST uncon...	50	5833	7095	7716	7745	8004	1	8004	20.00%	6.0/sec	6.22	1.10
TOTAL	50	5833	7095	7716	7745	8004	1	8004	20.00%	6.0/sec	6.22	1.10

☐ Include group name in label? ☒ Save Table Header

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
 (venv) PS C:\Users\Adala\Desktop\pro\ParPro->
 (venv) PS C:\Users\Adala\Desktop\pro\ParPro->

بعد تطبيق Thread Pool، أصبح عدد عمليات الدفع المتزامنة محدوداً بعدد العمال، أي 5 عمليات فقط في نفس الوقت.

```

[2026-05-18 21:06:51] CPU: 2.5% | RAM: 75.0%
[2026-05-18 21:06:52] CPU: 3.3% | RAM: 75.0%
[2026-05-18 21:06:53] CPU: 5.7% | RAM: 75.0%
[2026-05-18 21:06:54] CPU: 3.6% | RAM: 75.0%
[2026-05-18 21:06:55] CPU: 4.4% | RAM: 75.0%
[2026-05-18 21:06:56] CPU: 3.6% | RAM: 75.0%
[2026-05-18 21:06:57] CPU: 3.1% | RAM: 75.0%
[2026-05-18 21:06:58] CPU: 2.0% | RAM: 75.0%
[2026-05-18 21:06:59] CPU: 4.3% | RAM: 75.0%
[2026-05-18 21:07:00] CPU: 7.7% | RAM: 75.0%
[2026-05-18 21:07:01] CPU: 3.3% | RAM: 75.0%
[2026-05-18 21:07:02] CPU: 2.3% | RAM: 75.0%
[2026-05-18 21:07:03] CPU: 2.1% | RAM: 75.0%
[2026-05-18 21:07:04] CPU: 3.2% | RAM: 75.0%
[2026-05-18 21:07:05] CPU: 4.4% | RAM: 75.0%
[2026-05-18 21:07:06] CPU: 3.1% | RAM: 75.0%
[2026-05-18 21:07:07] CPU: 4.0% | RAM: 75.0%
[2026-05-18 21:07:08] CPU: 3.4% | RAM: 74.9%
[2026-05-18 21:07:09] CPU: 5.7% | RAM: 75.0%
[2026-05-18 21:07:10] CPU: 3.8% | RAM: 75.1%
[2026-05-18 21:07:11] CPU: 5.3% | RAM: 75.1%
[2026-05-18 21:07:12] CPU: 2.9% | RAM: 74.1%
[2026-05-18 21:07:13] CPU: 5.0% | RAM: 74.3%
[2026-05-18 21:07:14] CPU: 5.2% | RAM: 74.3%

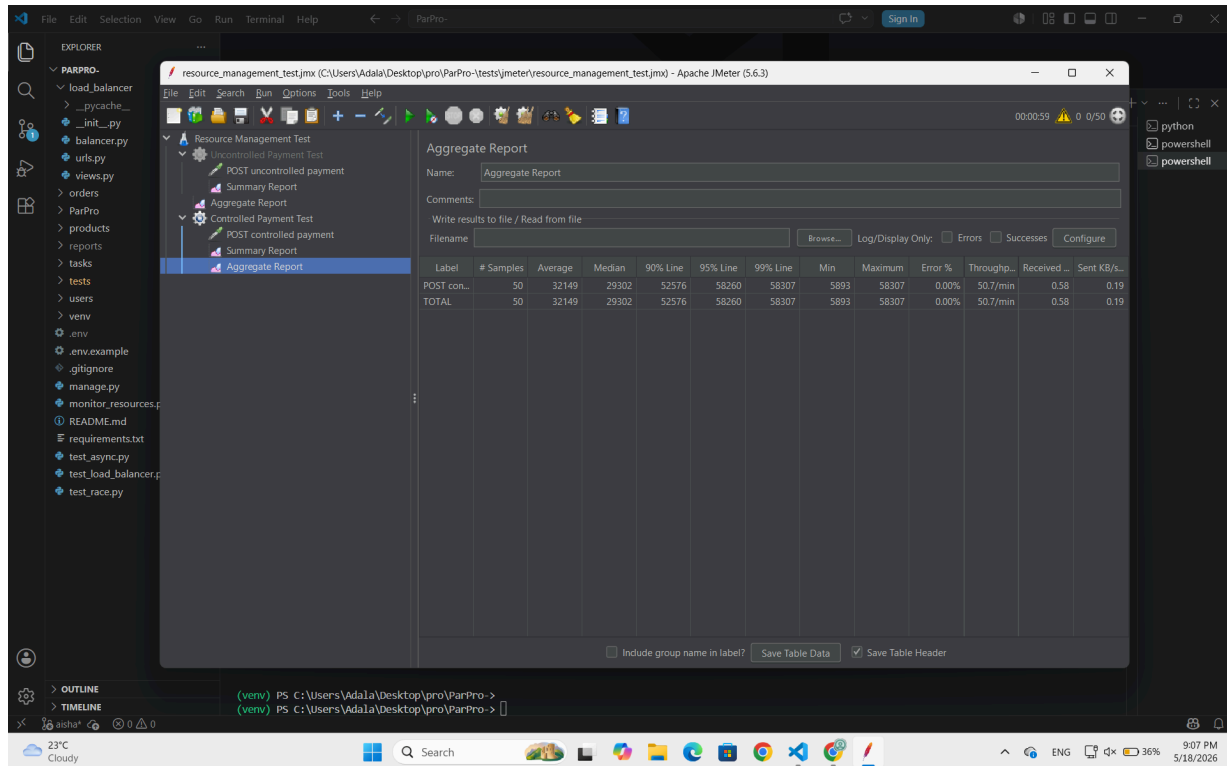
Monitoring stopped.
Results saved to: reports/resource_controlled_50_20260518_210607.csv
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> (Invoke-WebRequest -Uri "http://127.0.0.1:8000/orders/payment-metrics/" -Method GET).metrics

uncontrolled_active      : 0
uncontrolled_max_active  : 0
uncontrolled_total_requests : 0
controlled_active         : 0
controlled_max_active     : 5
controlled_total_requests : 50

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->

```

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
POST contr...	50	32149	5893	58307	16714.09	0.00%	50.7/min	0.58	0.19	708.9
TOTAL	50	32149	5893	58307	16714.09	0.00%	50.7/min	0.58	0.19	708.9



توضح النتائج أن الحل لم يمنع الطلبات من التنفيذ، لكنه نظم طريقة تنفيذ العمليات الثقيلة. في الحالة المضبوطة قد يزيد زمن الاستجابة لبعض الطلبات لأن بعضها ينتظر توفر worker، لكن بالمقابل يبقى استهلاك الموارد تحت السيطرة.

3. Asynchronous Processing

توجد في أنظمة التجارة الإلكترونية مهام ثانوية لا يجب أن تؤثر استجابة المستخدم، مثل إرسال إيميل تأكيد الطلب أو توليد الفاتورة. إذا تم تنفيذ هذه المهام داخل نفس مسار الطلب، فإن المستخدم ينتظر حتى تنتهي جميع المهام قبل الحصول على الرد.

في المشروع تم تنفيذ حالتين:

/orders/place – order – sync/

/orders/place – order – async/

في الحالة المتزامنة **place-order-sync** يتم إنشاء الطلب، ثم ينتظر النظام إرسال الإيميل وتوليد الفاتورة. تمت محاكاة إرسال الإيميل بزمان 2 ثانية، وتوليد الفاتورة بزمان 3 ثوان، لذلك يكون زمن الاستجابة تقريبا 5 ثوان.

في الحالة غير المتزامنة **place-order-async** يتم إنشاء الطلب بسرعة، ثم يتم وضع مهام الإيميل والفاتورة داخل Queue باستخدام **queue.Queue**. بعد ذلك يحصل المستخدم على الاستجابة مباشرة، بينما يقوم Worker في الخلفية بتنفيذ المهام.

تم اختيار **queue.Queue** كحل داخلي مبسط ومناسب لبيئة المشروع. في بيئة إنتاج حقيقية يمكن استبداله بحلول أقوى مثل Redis/Celery أو RabbitMQ.

تم اختبار الحالتين باستخدام الملف:

test_async.py

من خلال الأمر:

python test_async.py compare

أظهرت النتائج أن متوسط زمن الاستجابة في الحالة المتزامنة كان حوالي 5 ثوان، بينما انخفض في الحالة غير المتزامنة إلى أقل من ثانية. هذا يوضح أن نقل المهام الثانوية إلى الخلفية حسن زمن استجابة المستخدم بشكل واضح.

```
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python test_async.py compare

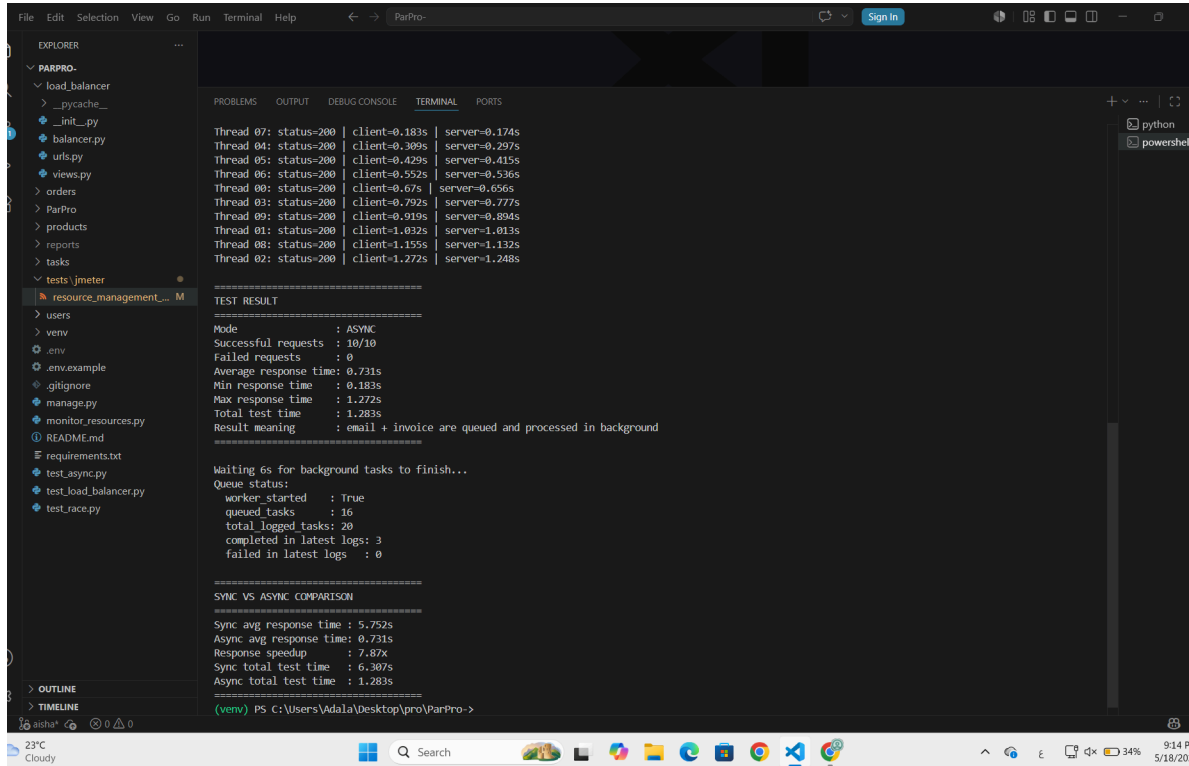
=====
Async Processing Test: SYNC
URL: http://127.0.0.1:8000/orders/place-order-sync/
Concurrent requests: 10
Initial stock: 50
=====

Thread 09: status=200 | client=5.21s | server=5.194s
Thread 01: status=200 | client=5.323s | server=5.303s
Thread 05: status=200 | client=5.448s | server=5.427s
Thread 06: status=200 | client=5.566s | server=5.548s
Thread 04: status=200 | client=5.687s | server=5.666s
Thread 00: status=200 | client=5.808s | server=5.786s
Thread 07: status=200 | client=5.93s | server=5.913s
Thread 02: status=200 | client=6.059s | server=6.03s
Thread 03: status=200 | client=6.182s | server=6.151s
Thread 08: status=200 | client=6.303s | server=6.27s

=====
TEST RESULT
=====
Mode          : SYNC
Successful requests : 10/10
Failed requests   : 0
Average response time: 5.752s
Min response time : 5.21s
Max response time : 6.303s
Total test time  : 6.307s
Result meaning   : user waits for email + invoice before response

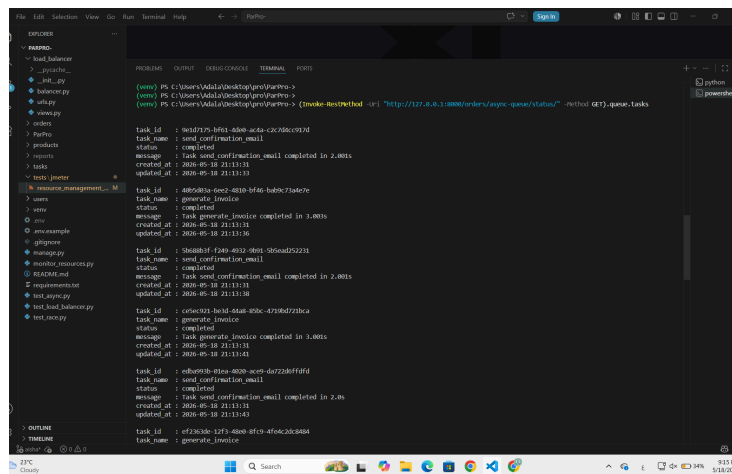
=====
Async Processing Test: ASYNC
URL: http://127.0.0.1:8000/orders/place-order-async/
Concurrent requests: 10
Initial stock: 50
=====

Thread 07: status=200 | client=0.183s | server=0.174s
```



كما تم توفير endpoint لمتابعة حالة الطابور:

/orders/async-queue/status/



توضح حالة الطابور أن مهام مثل `send_confirmation_email` و `generate_invoice` تم تنفيذها في الخلفية بعد إرجاع الاستجابة للمستخدم.

4. Batch Processing

في أنظمة التجارة الإلكترونية قد يحتاج النظام إلى تنفيذ عمليات دورية على عدد كبير من البيانات، مثل جرد المبيعات اليومية. معالجة كل الطلبات دفعة واحدة أو بشكل متسلسل قد لا تكون مناسبة مع زيادة حجم البيانات.

في هذا المشروع تم تطبيق Batch Processing على جرد المبيعات اليومية. تعتمد الفكرة على قراءة الطلبات المكتملة وتقسيمها إلى دفعات Chunks، ثم معالجة كل دفعة بشكل مستقل. تم أيضاً استخدام عدد من workers لمعالجة الدفعات بكفاءة أكبر.

تم تنفيذ عدة أوامر Django لإدارة هذا المتطلب:

```
seed_batch_orders.py
```

```
compare_daily_sales_processing.py
```

```
run_daily_sales_batch.py
```

أولاً تم توليد بيانات اختبار باستخدام:

```
python manage.py seed_batch_orders --count 1000
```

بعد ذلك تم تنفيذ مقارنة بين المعالجة المتسلسلة والمعالجة على دفعات باستخدام:

```
python manage.py compare_daily_sales_processing --chunk-size 100 --workers 4  
--simulate-ms 2
```

أظهرت المقارنة أن النتائج الحسابية بقيت متطابقة، مثل إجمالي الإيرادات وعدد الطلبات، لكن المعالجة على دفعات أعطت زمن تنفيذ أفضل عند وجود عمل محاكي لكل طلب. كما ظهرت قيمة Speedup التي توضح التحسن في الأداء.

بعد ذلك تم تشغيل Batch Job الحقيقي:

```
python manage.py run_daily_sales_batch --chunk-size 100 --workers 4
```

أظهر التشغيل أن 1000 طلب تمت معالجتها على 10 دفعات، كل دفعة تحتوي على 100 طلب، مع توليد تقرير CSV داخل مجلد `/reports`. كما تم تخزين نتائج التشغيل والدفعات داخل نماذج `BatchJobRun` و `BatchChunkLog`.

Visual Studio Code interface showing a terminal window with the following commands and output:

```
PS C:\Users\Adala\Desktop\pro\ParPro->
PS C:\Users\Adala\Desktop\pro\ParPro->
PS C:\Users\Adala\Desktop\pro\ParPro-> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PS C:\Users\Adala\Desktop\pro\ParPro-> .\venv\scripts\activate
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python manage.py seed_batch_orders --count 1000
Generating 1000 completed orders...
Created 500/1000 orders...
Created 1000/1000 orders...
Successfully generated 1000 completed orders in 0.95s.
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python manage.py compare_daily_sales_processing --chunk-size 100 --workers 4 --simulate-ms 2
```

Output:

```
=====
Daily Sales Processing Comparison
=====
Target date: 2026-05-18
Total completed orders: 1000
Chunk size: 100
Workers: 4
Simulated work per order: 2.0 ms
=====

NAIVE SEQUENTIAL PROCESSING
=====
Method: process all orders one by one without chunks or workers
Orders processed: 1000
Total revenue: 154240.00
Duration: 2.4844s

CHUNK-BASED BATCH PROCESSING
=====
Method: process 10 chunks using 4 workers
Chunk 01: orders=100 | revenue=14430.00 | duration=0.2522s
Chunk 04: orders=100 | revenue=14090.00 | duration=0.2566s
Chunk 02: orders=100 | revenue=15140.00 | duration=0.2575s
Chunk 03: orders=100 | revenue=15760.00 | duration=0.2588s
Chunk 05: orders=100 | revenue=13640.00 | duration=0.2505s
Chunk 07: orders=100 | revenue=15310.00 | duration=0.2490s
Chunk 08: orders=100 | revenue=15610.00 | duration=0.2510s
Chunk 06: orders=100 | revenue=16490.00 | duration=0.2527s
Chunk 09: orders=100 | revenue=17730.00 | duration=0.2443s
Chunk 10: orders=100 | revenue=16040.00 | duration=0.2417s
=====
```

Comparison Result:

```
=====
COMPARISON RESULT
=====
Naive revenue: 154240.00
Batch revenue: 154240.00
Revenue consistent: True
Naive duration: 2.4844s
Batch duration: 0.7538s
Speedup: 3.30x
Chunks processed: 10
Orders processed by batch: 1000
Comparison successful: batch processing produced consistent results.
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python manage.py run_daily_sales_batch --chunk-size 100 --workers 4
```

Output:

```
=====
Daily Sales Batch Processing
=====
Target date: 2026-05-18
Total completed orders: 1000
Chunk size: 100
Workers: 4
=====
Total chunks: 10
Chunk 1 finished | orders=100 | revenue=14430.00 | status=completed
Chunk 5 finished | orders=100 | revenue=13640.00 | status=completed
Chunk 10 finished | orders=100 | revenue=16490.00 | status=completed
Chunk 7 finished | orders=100 | revenue=15310.00 | status=completed
Chunk 8 finished | orders=100 | revenue=15610.00 | status=completed
Chunk 9 finished | orders=100 | revenue=17730.00 | status=completed
Chunk 10 finished | orders=100 | revenue=16040.00 | status=completed
Chunk 2 finished | orders=100 | revenue=15140.00 | status=completed
Chunk 3 finished | orders=100 | revenue=15760.00 | status=completed
Chunk 4 finished | orders=100 | revenue=14090.00 | status=completed
=====
Batch job completed successfully.
Job run id: 3
Processed orders: 1000
Total revenue: 154240.00
Duration: 0.5441s
CSV report: reports\daily_sales_batch_run_3.csv
=====
```

5. Load Distribution

عند زيادة عدد الطلبات على النظام، قد لا يكون تشغيل نسخة واحدة من التطبيق كافياً. لذلك يمكن توزيع الطلبات على أكثر من نسخة من التطبيق باستخدام Load Balancer. الهدف من هذا المتطلب هو إثبات أن الطلبات يمكن أن تتحول إلى أكثر من app instance، ومعرفة مقدار التوزيع بين النسخ.

في المشروع تم تنفيذ Application-Level Load Balancer داخل Django. يعمل ال Load Balancer على المنفذ 8000، ويوزع الطلبات على ثلاث نسخ من التطبيق تعمل على المنافذ التالية:

8001
8002
8003

تم تنفيذ عدة استراتيجيات للتوزيع:

Round Robin
Least Connections
IP Hash

لاختبار المتطلب، تم تشغيل أربع نسخ من Django في أربعة terminals :

```
python manage.py runserver 8000  
python manage.py runserver 8001  
python manage.py runserver 8002  
python manage.py runserver 8003
```

ثم تم تشغيل الاختبار باستخدام:

```
python test_load_balancer.py round_robin
```

أظهرت نتيجة Round Robin توزيعاً متساوياً للطلبات بين الخوادم الثلاثة، حيث حصل كل خادم على 10 طلبات من أصل 30 طلباً. هذا يثبت أن الطلبات تم توزيعها على أكثر من نسخة من التطبيق.


```
(venv) PS C:\Users\Adala\Desktop\pro\ParPro> python test_load_balancer.py round_robin
Reset stats: 200 | {"success": true, "message": "Load balancer stats reset successfully"}

Strategy: ROUND_ROBIN | Requests: 30

Thread 29 + server_1 | status: 200
Thread 13 + server_2 | status: 200
Thread 22 + server_3 | status: 200
Thread 23 + server_2 | status: 200
Thread 06 + server_1 | status: 200
Thread 20 + server_3 | status: 200
Thread 17 + server_2 | status: 200
Thread 14 + server_3 | status: 200
Thread 11 + server_1 | status: 200
Thread 12 + server_2 | status: 200
Thread 10 + server_3 | status: 200
Thread 08 + server_1 | status: 200
Thread 19 + server_3 | status: 200
Thread 04 + server_1 | status: 200
Thread 21 + server_2 | status: 200
Thread 02 + server_1 | status: 200
Thread 00 + server_3 | status: 200
Thread 03 + server_2 | status: 200
Thread 27 + server_2 | status: 200
Thread 25 + server_1 | status: 200
Thread 07 + server_3 | status: 200
Thread 16 + server_1 | status: 200
Thread 01 + server_2 | status: 200
Thread 24 + server_3 | status: 200
Thread 15 + server_2 | status: 200
Thread 18 + server_1 | status: 200
Thread 09 + server_3 | status: 200
Thread 28 + server_1 | status: 200
Thread 26 + server_3 | status: 200
Thread 05 + server_2 | status: 200

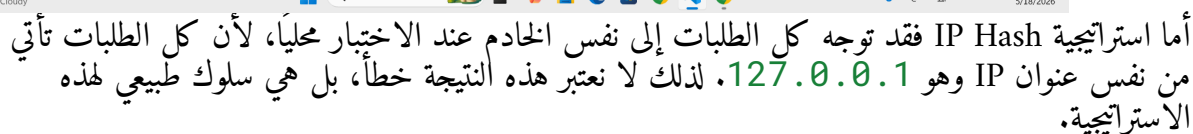
=====
DISTRIBUTION RESULT
=====
server_1: ██████████ (10 reqs - 33%)
server_2: ██████████ (10 reqs - 33%)
server_3: ██████████ (10 reqs - 33%)

=====
RESULT: BALANCED
Total handled: 30
=====
```

كما تم اختبار Least Connections:

`python test_load_balancer.py least_connections`

أظهرت هذه الاستراتيجية توزيعاً قريباً من التوازن، مع اختلاف بسيط حسب عدد الاتصالات النشطة لحظة إرسال الطلب.



أدوات الاختبار المستخدمة

تم استخدام عدة أدوات لاختبار المتطلبات غير الوظيفية حسب طبيعة كل متطلب. في بعض المتطلبات تم استخدام سكريبتات Python لمحاكاة الطلبات المتزامنة، وفي متطلب إدارة الموارد تم استخدام JMeter لأنه مناسب لاختبارات الضغط وإرسال عدد كبير من الطلبات.

الخاتمة

في هذا المشروع تم تطبيق خمسة متطلبات غير وظيفية على نظام تجارة إلكترونية مبسط باستخدام Django و PostgreSQL. تم ربط كل متطلب بسيناريو عملي داخل المشروع، مثل شراء منتج من المخزون، معالجة الدفع، إرسال الإيميل، توليد الفاتورة، جرد المبيعات، وتوزيع الطلبات على أكثر من خادم.

أثبتت النتائج أن استخدام المعاملات وقفل الصفوف ساعد على حل مشكلة Race Condition، وأن استخدام Thread Pool ساعد على التحكم بعدد العمليات الثقيلة المتزامنة. كما أظهرت المعالجة غير المتزامنة تحسناً واضحاً في زمن الاستجابة، بينما ساعد Batch Processing على تنظيم معالجة عدد كبير من الطلبات. أخيراً، أظهر Load Distribution أن الطلبات يمكن توزيعها على أكثر من نسخة من التطبيق بدلاً من الاعتماد على نسخة واحدة فقط.

بشكل عام، يوضح المشروع كيف يمكن تحسين جودة النظام ليس فقط من خلال الوظائف التي يقدمها، بل أيضاً من خلال طريقة تعامله مع الضغط، التزامن، الموارد، والمهام الخلفية. لذلك يمكن اعتبار المشروع نموذجاً مبسطاً لتطبيق مفاهيم مهمة تستخدم في الأنظمة الخلفية الحقيقية.

References

- [1] Django Documentation, "Database transactions"
- [2] Django Documentation, "QuerySet select_for_update"
- [3] Python Documentation, "queue — A synchronized queue class"
- [4] Python Documentation, "concurrent.futures — ThreadPoolExecutor"
- [5] Apache JMeter Documentation, "User Manual"
- [6] PostgreSQL Documentation, "Explicit Locking"